

Cybersecurity Cypher Queries over AWS-native resources on Arbitium's Graph DB

Assumption 1

Nodes use universal labels like :User, :Role, :Policy, :Function, :Compute, :API, :DataStore, :ObjectStorage, :SecurityBoundary, and AWS-native metadata in properties like provider_type, name, arn, actions, cidr, ports, public, environment, etc.

The ontology supports universal node types and normalized relationships such as GRANTS, ASSUMES, HAS_POLICY, READS, WRITES, ROUTES_TO, PROTECTED_BY, and DEPLOYED_IN; the AWS dictionary maps native AWS resources like IAM users, roles, policies, Lambda, EC2, DynamoDB, RDS, S3, Route53, security groups, and KMS into those universal types.

1. Public DNS entry ? API ? Lambda ? privileged role

```
MATCH path =
  (dns:DNSRecord)-[:ROUTES_TO]->(api:API)-[:INVOKES|INTEGRATES_WITH]->(fn:Function)-[:ASSUMES|ASSUME
  S_ROLE]->(role:Role)
MATCH (role)-[:HAS_POLICY]->(policy:Policy)-[g:GRANTS]->(target)
WHERE dns.public = true
  AND any(action IN g.actions WHERE action ENDS WITH '.*' OR action = '*')
RETURN dns.name, api.name, fn.name, role.name, policy.name, g.actions, target.name, path;
```

Finds internet-reachable application paths that end in broad IAM permissions. This is a good ?external entrypoint to privilege? query.

2. Internet-facing APIs without WAF or certificate protection

```
MATCH (api:API)
WHERE api.public = true
  AND NOT (api)-[:PROTECTED_BY]->(:SecurityBoundary {provider_type: 'waf_web_acl'})
  AND NOT (api)-[:PROTECTED_BY]->(:Certificate)
RETURN api.name, api.provider_type, api.domain_name, api.region;
```

Looks for exposed APIs, load balancers, or CloudFront/API Gateway surfaces missing expected edge protection.

3. Compute reachable from public DNS and able to write sensitive data

```
MATCH path =
  (:DNSRecord {public: true})-[:ROUTES_TO|FORWARDS_TO|INVOKES|INTEGRATES_WITH*1..4]->(workload)
MATCH (workload)-[:WRITES]->(data)
WHERE workload:Function OR workload:Compute
  AND (data:DataStore OR data:ObjectStorage)
  AND coalesce(data.classification, '') IN ['sensitive', 'pii', 'confidential', 'regulated']
RETURN workload.name, labels(workload), data.name, data.classification, path;
```

Identifies public ingress paths that can mutate sensitive storage. This is useful for blast-radius review.

4. IAM principals with wildcard grants

```
MATCH (principal)-[:HAS_POLICY|MEMBER_OF*0..2]->(p:Policy)-[g:GRANTS]->(target)
WHERE principal:User OR principal:Role OR principal:Group
    AND any(action IN g.actions WHERE action = '*' OR action ENDS WITH ':*')
RETURN principal.name, labels(principal), p.name, g.actions, target.name, labels(target)
ORDER BY principal.name;
```

Finds users, groups, and roles that inherit broad permissions through direct or indirect policy attachment. IAM policy discovery is modeled as GRANTS authorization edges in the AWS subontology.

5. Human users with direct access to secrets or KMS keys

```
MATCH (u:User)-[:HAS_POLICY|MEMBER_OF|HAS_MEMBER*0..3]->(p:Policy)-[g:GRANTS]->(s:Secret)
WHERE any(action IN g.actions WHERE
    action IN ['secretsmanager:GetSecretValue', 'ssm:GetParameter', 'ssm:GetParameters',
'kms:Decrypt']
    OR action = '*'
    OR action ENDS WITH ':*')
RETURN u.name, p.name, g.actions, s.name, s.provider_type;
```

Looks for human identity access to secrets, parameters, or KMS decrypt permissions.

6. Lambda functions with data access but no VPC boundary

```
MATCH (fn:Function)-[:READS|WRITES]->(data)
WHERE data:DataStore OR data:ObjectStorage
    AND NOT (fn)-[:RUNS_IN|DEPLOYED_IN]->(:Network)
RETURN fn.name, collect(DISTINCT data.name) AS data_accessed;
```

Finds serverless workloads touching data while not attached to a VPC. That is not always wrong, but it is worth reviewing for sensitive workloads.

7. Workloads sharing the same overprivileged role

```
MATCH (w)-[:ASSUMES|ASSUMES_ROLE]->(role:Role)
WHERE w:Function OR w:Compute
WITH role, collect(w) AS workloads
WHERE size(workloads) > 1
MATCH (role)-[:HAS_POLICY]->(p:Policy)-[g:GRANTS]->(target)
WHERE any(action IN g.actions WHERE action = '*' OR action ENDS WITH ':*')
RETURN role.name, [w IN workloads | w.name] AS workloads, p.name, g.actions, target.name;
```

Finds role reuse across workloads, especially dangerous when the shared role has broad permissions.

8. Public subnet workloads with database access

```
MATCH (w)-[:DEPLOYED_IN]->(subnet:Subnet)-[:MEMBER_OF|BELONGS_TO]->(vpc:Network)
MATCH (w)-[:READS|WRITES]->(db:DataStore)
WHERE (w:Compute OR w:Function)
    AND subnet.public = true
RETURN w.name, labels(w), subnet.name, vpc.name, db.name;
```

Shows compute in public subnets that can access databases. This is a classic exposure + lateral movement check.

9. Databases exposed through permissive security boundaries

```
MATCH (db:DataStore)-[:MEMBER_OF|PROTECTED_BY]->(sg:SecurityBoundary)
WHERE sg.provider_type = 'security_group'
      AND any(rule IN sg.ingress_rules WHERE rule.cidr = '0.0.0.0/0')
RETURN db.name, db.provider_type, sg.name, sg.ingress_rules;
```

Finds RDS, OpenSearch, ElastiCache, or similar data stores protected by public ingress rules. The AWS subontology maps database resources to VPCs/security groups and KMS where available.

10. S3 buckets with write access from public-facing workloads

```
MATCH ingress =
  (:DNSRecord {public: true})-[:ROUTES_TO|FORWARDS_TO|INVOKES|INTEGRATES_WITH*1..4]->(w)
MATCH (w)-[:WRITES]->(bucket:ObjectStorage)
WHERE w:Function OR w:Compute
RETURN w.name, labels(w), bucket.name, ingress;
```

Finds public request paths that can write to object storage. Useful for upload abuse, data poisoning, and persistence analysis.

11. Sensitive resources without KMS protection

```
MATCH (r)
WHERE (r:DataStore OR r:ObjectStorage)
      AND coalesce(r.classification, '') IN ['sensitive', 'pii', 'confidential', 'regulated']
      AND NOT (r)-[:PROTECTED_BY|USES_KEY]->(:Secret {provider_type: 'kms_key'})
RETURN r.name, labels(r), r.provider_type, r.classification;
```

Finds sensitive databases or buckets missing explicit KMS/key protection edges. The dictionary maps KMS keys as Secret resources and DynamoDB/S3 protection relationships through PROTECTED_BY.

12. Privilege escalation: user ? group ? policy ? role assumption

```
MATCH path =
  (u:User)-[:MEMBER_OF]->(g:Group)-[:HAS_POLICY]->(p:Policy)-[grant:GRANTS]->(role:Role)
WHERE any(action IN grant.actions WHERE action = 'sts:AssumeRole' OR action = '')
RETURN u.name, g.name, p.name, grant.actions, role.name, path;
```

Finds users who can assume roles through group-inherited permissions.

13. Cross-account trust or access

```
MATCH (src)-[r:GRANTS|TRUSTS|ASSUMES|CONNECTS_TO|CONNECTS]->(dst)
WHERE src.account_id IS NOT NULL
      AND dst.account_id IS NOT NULL
      AND src.account_id <> dst.account_id
RETURN type(r) AS relationship, src.name, labels(src), src.account_id,
      dst.name, labels(dst), dst.account_id, r.actions;
```

Shows cross-account trust, peering, grants, and assume-role paths. This is useful for tenant/account boundary review.

14. Orphaned public DNS records

```
MATCH (dns:DNSRecord)
WHERE dns.public = true
  AND NOT (dns)-[:ROUTES_TO]->()
RETURN dns.name, dns.record_type, dns.value, dns.zone;
```

Finds public DNS entries that no longer route to modeled infrastructure. These can indicate stale records, takeover risk, or incomplete scanning.

15. Data exfiltration path from public endpoint to external integration

```
MATCH path =
  (:DNSRecord {public:
true})-[:ROUTES_TO|INVOKES|INTEGRATES_WITH|TRIGGERS|WRITES|DELIVERS_TO*1..6]->(sink)
WHERE sink:Queue OR sink:Topic OR sink:Stream OR sink:DeliveryStream OR sink:Workflow
RETURN sink.name, labels(sink), path;
```

Finds public-triggerable paths that eventually write to queues, topics, streams, delivery streams, or workflows. This helps detect async exfiltration or abuse routes.

16. Workloads that can read secrets and write externally

```
MATCH (w)-[:READS|GRANTS]->(secret:Secret)
MATCH path = (w)-[:WRITES|DELIVERS_TO|STREAMS_TO|ROUTES_TO*1..3]->(out)
WHERE w:Function OR w:Compute
  AND (out:Queue OR out:Topic OR out:Stream OR out:ObjectStorage)
RETURN w.name, secret.name, out.name, labels(out), path;
```

Looks for workloads that combine secret access with outbound write capability.

17. Resources with no owner tag

```
MATCH (r:InfrastructureObject)
WHERE NOT (:Tag {key: 'owner'})-[:ASSOCIATED_WITH]->(r)
  AND NOT (:Tag {key: 'team'})-[:ASSOCIATED_WITH]->(r)
RETURN labels(r), r.name, r.provider_type, r.account_id, r.region;
```

Finds unowned infrastructure. From a security operations perspective, unknown ownership increases incident response time.

18. Critical resources without audit/log coverage

```
MATCH (r)
WHERE r:API OR r:Function OR r:Compute OR r:DataStore OR r:ObjectStorage
  AND NOT (:AuditTrail)-[:WRITES|OBSERVES|EMITS]->(r)
  AND NOT (:LogStore)-[:EMITS|OBSERVES]->(r)
RETURN labels(r), r.name, r.provider_type, r.account_id, r.region;
```

Finds critical infrastructure without modeled audit/log observability. The ontology includes observability resources such as AuditTrail, LogStore, Alert, and relationships

like WRITES, OBSERVES, and EMITS.

19. Internet gateway path to private data workloads

```
MATCH path =
  (igw {provider_type: 'internet_gateway'})-[:ATTACHED_TO|CONNECTS|ROUTES_TO|ROUTES_THROUGH|ASSOCIATED_WITH*1..5]->(db:DataStore)
RETURN db.name, db.provider_type, path;
```

Looks for routable paths from internet gateways toward data stores. Depending on how route tables are modeled, this can reveal accidental exposure.

20. Full attack path: public DNS ? app ? compute ? role ? policy ? data

```
(dns:DNSRecord {public: true})
-[:ROUTES_TO|FORWARDS_TO|INVOKES|INTEGRATES_WITH*1..3]->
(w)
-[:ASSUMES|ASSUMES_ROLE]->
(role:Role)
-[:HAS_POLICY]->
(policy:Policy)
-[:GRANTS]->
(data)
WHERE (w:Function OR w:Compute)
AND (data:DataStore OR data:ObjectStorage OR data:Secret)
RETURN dns.name, w.name, role.name, policy.name, data.name, labels(data), path;
```

This is the “show me the attack story” query: public endpoint, executable workload, assumed role, policy, and final target. It is the kind of query that turns the graph into an attack-path map.

BLAST RADIUS QUERIES

1. Forward blast radius from a compromised workload

```
MATCH path =
  (start {id: $resource_id})
-[:ASSUMES|HAS_POLICY|GRANTS|READS|WRITES|INVOKES|TRIGGERS|ROUTES_TO|DELIVERS_TO|STREAMS_TO*1..5]->
  (affected)
RETURN affected.name,
  labels(affected) AS affected_type,
  affected.provider_type,
  length(path) AS distance,
  path
ORDER BY distance ASC;
```

Shows everything a compromised Lambda, EC2 instance, container, API, or role could reach through execution, permission, data, and integration paths.

2. Blast radius summary by resource domain

```
MATCH path =
  (start {id: $resource_id})
-[:ASSUMES|HAS_POLICY|GRANTS|READS|WRITES|INVOKES|TRIGGERS|ROUTES_TO|DELIVERS_TO|STREAMS_TO*1..5]->
  (affected)
WITH DISTINCT affected
RETURN affected.functional_domain AS domain,
```

```
count(*) AS affected_count,
collect(affected.name)[0..20] AS examples
ORDER BY affected_count DESC;
```

Turns a raw blast radius into a domain-level summary: identity, compute, data, storage, network, integration, observability, platform, etc.

3. Sensitive-data blast radius

```
MATCH path =
  (start {id: $resource_id})
  -[:ASSUMES|HAS_POLICY|GRANTS|READS|WRITES|INVOKES|TRIGGERS*1..5]->
  (data)
WHERE data:DataStore OR data:ObjectStorage OR data:Secret
  AND coalesce(data.classification, '') IN ['pii', 'sensitive', 'confidential', 'regulated']
RETURN data.name,
       labels(data) AS data_type,
       data.classification,
       length(path) AS distance,
       path
ORDER BY distance ASC;
```

Answers: ?If this node is compromised, what sensitive data can be reached??

4. Write-capability blast radius

```
MATCH path =
  (start {id: $resource_id})
  -[:ASSUMES|HAS_POLICY|GRANTS|WRITES|WRITES_TO|DELIVERS_TO|STREAMS_TO*1..5]->
  (target)
WHERE target:DataStore
  OR target:ObjectStorage
  OR target:Queue
  OR target:Topic
  OR target:Stream
  OR target:DeliveryStream
RETURN target.name,
       labels(target) AS target_type,
       target.provider_type,
       path;
```

Finds where an attacker could modify state, poison queues, alter data, or push outbound messages.

5. Reverse blast radius for a compromised datastore

```
MATCH path =
  (dependent)
  -[:READS|WRITES|READS_FROM|WRITES_TO|USES|STORES_IN|GRANTS*1..4]->
  (data {id: $datastore_id})
RETURN dependent.name,
       labels(dependent) AS dependent_type,
       dependent.provider_type,
       length(path) AS distance,
       path
ORDER BY distance ASC;
```

Shows which workloads, APIs, workflows, or identities depend on a datastore. Useful for incident containment and outage impact.

6. Role compromise blast radius

```
MATCH path =
  (role:Role {id: $role_id})
  -[:HAS_POLICY|GRANTS|READS|WRITES|USES|INVOKES|TRIGGERS*1..4]->
  (affected)
RETURN affected.name,
       labels(affected) AS affected_type,
       affected.provider_type,
       path;
```

Calculates what a stolen IAM role can affect. The AWS dictionary maps Lambda/EC2 role assumption, policy attachment, and policy grants into universal ASSUMES, HAS_POLICY, and GRANTS edges.

7. Workloads affected by one compromised role

```
MATCH (workload)-[:ASSUMES]->(role:Role {id: $role_id})
WHERE workload:Function OR workload:Compute
RETURN role.name,
       count(workload) AS affected_workload_count,
       collect(workload.name) AS affected_workloads;
```

Answers: ?How many runtimes share this identity?? This reveals role reuse risk.

8. Policy blast radius

```
MATCH path =
  (principal)-[:HAS_POLICY]->(policy:Policy {id: $policy_id})-[:GRANTS]->(target)
WHERE principal:User OR principal:Group OR principal:Role
RETURN principal.name,
       labels(principal) AS principal_type,
       target.name,
       labels(target) AS target_type,
       path;
```

Shows every principal and target affected by a single policy.

9. Wildcard policy blast radius

```
MATCH (principal)-[:HAS_POLICY]->(policy:Policy)-[g:GRANTS]->(target)
WHERE any(action IN g.actions WHERE action = '*' OR action ENDS WITH '.*')
RETURN policy.name,
       principal.name,
       labels(principal) AS principal_type,
       target.name,
       labels(target) AS target_type,
       g.actions
ORDER BY policy.name;
```

Finds policies whose compromise or misconfiguration creates broad systemic impact.

10. KMS key compromise blast radius

```
MATCH path =
  (resource)-[:USES_KEY|PROTECTED_BY]->(key:Secret {id: $kms_key_id})
RETURN key.name,
       resource.name,
       labels(resource) AS resource_type,
       resource.provider_type,
       resource.classification,
       path
```

```
ORDER BY resource.classification DESC;
```

Shows all buckets, databases, secrets, log groups, or other resources relying on one KMS key. The AWS subontology includes USES_KEY relationships for secrets, log groups, ECS, RDS, and related resources.

11. Security group blast radius

```
MATCH (resource)-[:MEMBER_OF|PROTECTED_BY]->(sg:SecurityBoundary {id: $security_group_id})
RETURN sg.name,
       count(resource) AS protected_resource_count,
       collect({
         name: resource.name,
         type: labels(resource),
         provider_type: resource.provider_type
       }) AS protected_resources;
```

Answers: ?What breaks or becomes exposed if this security group is changed??

12. Public security boundary blast radius

```
MATCH (resource)-[:MEMBER_OF|PROTECTED_BY]->(sg:SecurityBoundary)
WHERE any(rule IN sg.ingress_rules WHERE rule.cidr = '0.0.0.0/0' OR rule.cidr = ':::/0')
RETURN sg.name,
       sg.ingress_rules,
       count(resource) AS exposed_resource_count,
       collect(resource.name) AS exposed_resources
ORDER BY exposed_resource_count DESC;
```

Shows how many resources inherit public exposure from permissive security boundaries.

13. Subnet compromise blast radius

```
MATCH path =
  (subnet:Subnet {id: $subnet_id})<-[:DEPLOYED_IN|IN_SUBNET|HOSTS]-(resource)
OPTIONAL MATCH downstream =
  (resource)-[:READS|WRITES|ASSUMES|INVOKES|TRIGGERS|ROUTES_TO*1..3]->(affected)
RETURN subnet.name,
       resource.name AS directly_affected,
       labels(resource) AS direct_type,
       collect(DISTINCT affected.name) AS downstream_affected;
```

Calculates both direct subnet residents and second-order impact through their permissions and dependencies.

14. VPC/network blast radius

```
MATCH path =
  (vpc:Network {id: $network_id})<-[:BELONGS_TO|DEPLOYED_IN|MEMBER_OF|CONTAINS*1..3]-(resource)
RETURN vpc.name,
       labels(resource) AS resource_type,
       resource.provider_type,
       count(resource) AS count,
       collect(resource.name)[0..25] AS examples
ORDER BY count DESC;
```

Answers: ?What infrastructure is inside this network boundary??

15. API compromise blast radius

```
MATCH path =
  (api:API {id: $api_id})
  -[:INVOKES|INTEGRATES_WITH|ROUTES_TO|FORWARDS_TO|TRIGGERS*1..5]->
  (affected)
RETURN api.name,
       affected.name,
       labels(affected) AS affected_type,
       affected.provider_type,
       length(path) AS distance,
       path
ORDER BY distance ASC;
```

Shows downstream compute, workflows, queues, functions, and data paths reachable from one compromised API.

16. Public endpoint blast radius

```
MATCH path =
  (entry)
  -[:ROUTES_TO|FORWARDS_TO|INVOKES|INTEGRATES_WITH|TRIGGERS|READS|WRITES|ASSUMES|GRANTS*1..6]->
  (affected)
WHERE (entry:DNSRecord OR entry:API)
      AND entry.public = true
RETURN entry.name,
       labels(entry) AS entry_type,
       affected.name,
       labels(affected) AS affected_type,
       length(path) AS distance,
       path
ORDER BY entry.name, distance;
```

Measures blast radius starting from anything internet-facing.

17. Shared dependency blast radius

```
MATCH (dependent)-[:USES|READS|WRITES|INVOKES|DEPLOYED_IN|PROTECTED_BY|USES_KEY]->(shared)
WITH shared, collect(DISTINCT dependent) AS dependents
WHERE size(dependents) >= $min_dependents
RETURN shared.name,
       labels(shared) AS shared_type,
       shared.provider_type,
       size(dependents) AS dependent_count,
       [d IN dependents | d.name][0..50] AS examples
ORDER BY dependent_count DESC;
```

Finds ?choke points?: shared keys, roles, security groups, buckets, queues, VPCs, subnets, or APIs.

18. Integration cascade blast radius

```
MATCH path =
  (start {id: $resource_id})
  -[:TRIGGERS|DELIVERS_TO|STREAMS_TO|WRITES_TO|INVOKES*1..6]->
  (integration)
WHERE integration:Queue
      OR integration:Topic
      OR integration:EventBus
      OR integration:Stream
      OR integration:DeliveryStream
      OR integration:Workflow
RETURN integration.name,
```

```

    labels(integration) AS integration_type,
    length(path) AS distance,
    path
ORDER BY distance ASC;

```

Shows how a compromised resource can propagate through event-driven systems.

19. Alerting/observability blast radius

```

MATCH path =
  (observed {id: $resource_id})<-[:OBSERVES|EMITS|WRITES_TO|STREAMS_TO|DELIVERS_TO*1..4]- (obs)
WHERE obs:LogStore OR obs:AuditTrail OR obs:Alert OR obs:Metric OR obs:Dashboard
RETURN observed.name,
       obs.name,
       labels(obs) AS observability_type,
       path;

```

Answers: ?If this resource is compromised, what telemetry should exist?? or ?If this log pipeline fails, what detection coverage is lost??

20. Account-level blast radius by criticality

```

MATCH (account:Account {id: $account_id})-[:CONTAINS*1..5]->(resource)
RETURN resource.criticality AS criticality,
       resource.functional_domain AS domain,
       count(resource) AS count,
       collect(resource.name)[0..20] AS examples
ORDER BY criticality DESC, count DESC;

```

Gives an account-wide impact inventory grouped by business/security criticality.

21. Cross-account blast radius

```

MATCH path =
  (start {account_id: $account_id})
  -[:TRUSTS|ASSUMES|GRANTS|CONNECTS|CONNECTS_TO|ROUTES_TO|ROUTES_THROUGH*1..5]->
  (external)
WHERE external.account_id IS NOT NULL
      AND external.account_id <> $account_id
RETURN external.account_id,
       external.name,
       labels(external) AS external_type,
       length(path) AS distance,
       path
ORDER BY external.account_id, distance;

```

Finds how compromise can cross account boundaries through trust, network connectivity, routing, or authorization.

22. ?Crown jewel? shortest paths from public entrypoints

```

MATCH (entry)
WHERE (entry:DNSRecord OR entry:API)
      AND entry.public = true
MATCH (crown)
WHERE crown.criticality = 'critical'
      OR crown.classification IN ['pii', 'regulated', 'confidential']
MATCH path = shortestPath(
  (entry)-[:ROUTES_TO|FORWARDS_TO|INVOKES|INTEGRATES_WITH|ASSUMES|GRANTS|READS|WRITES|TRIGGERS*1..8]
->(crown)
)
RETURN entry.name,

```

```

    crown.name,
    labels(crown) AS crown_type,
    length(path) AS distance,
    path
ORDER BY distance ASC;

```

This is one of the most important blast-radius queries: ?How close are the crown jewels to the internet??

23. Top 20 highest-blast-radius identities

```

MATCH path =
  (principal)-[:HAS_POLICY|GRANTS|ASSUMES|READS|WRITES|INVOKES|TRIGGERS*1..5]->(affected)
WHERE principal:User OR principal:Role OR principal:Group
WITH principal, count(DISTINCT affected) AS blast_count
RETURN principal.name,
       labels(principal) AS principal_type,
       principal.provider_type,
       blast_count
ORDER BY blast_count DESC
LIMIT 20;

```

Ranks identities by how much they can affect.

24. Top 20 highest-blast-radius workloads

```

MATCH path =
  (workload)-[:ASSUMES|HAS_POLICY|GRANTS|READS|WRITES|INVOKES|TRIGGERS|DELIVERS_TO|STREAMS_TO*1..5]->(affected)
WHERE workload:Function OR workload:Compute
WITH workload, count(DISTINCT affected) AS blast_count
RETURN workload.name,
       labels(workload) AS workload_type,
       workload.provider_type,
       blast_count
ORDER BY blast_count DESC
LIMIT 20;

```

Ranks Lambda functions, EC2 instances, ECS services, or other workloads by potential impact.

25. Deletion/removal impact for a resource

```

MATCH path =
  (dependent)-[:USES|READS|WRITES|INVOKES|ROUTES_TO|DEPLOYED_IN|PROTECTED_BY|USES_KEY|AUTHENTICATES]->(target {id: $resource_id})
RETURN target.name,
       dependent.name,
       labels(dependent) AS dependent_type,
       type(last(relationships(path))) AS dependency_type,
       path;

```

Useful for change management: ?What will break if this resource is deleted, rotated, detached, or disabled??